

QA TECHNICAL EVALUATION

TEST PLAN

Prepared by	Keresia
Application Under Test	AURIX
Evaluation Type	24-hour End-to-End QA Technical Evaluation
Evaluation Window	24 hours from receipt of evaluation
Primary URL	https://aurixapp.de
Document Status	Initial Test Plan

Purpose: To define a risk-based, structured approach for evaluating the functional quality, usability, API behavior, security controls, performance, regression stability, and basic automation coverage of the AURIX web application.

1. Test Plan Overview

This test plan defines the approach for the 24-hour HOPn/AURIX QA technical evaluation. The assessment will focus on critical customer journeys and high-risk financial workflows, while balancing breadth of coverage with the limited evaluation window. Testing will be risk-based and evidence-driven, with particular attention to authentication, authorization, wallet and investment transactions, product eligibility, portfolio consistency, API behavior, security-oriented checks, usability, responsiveness, and reporting quality.

2. Test Objectives

- Validate that critical user journeys function correctly from registration through logout.
- Verify that financial data, balances, transactions, portfolio values, and investment outcomes remain consistent across the application.
- Validate business rules for country-based ETF eligibility and subscription-based access.
- Assess API behavior for authentication, users, ETFs, portfolios, investments, transactions, and insights.
- Identify security-oriented defects such as authentication bypass, authorization weaknesses, IDOR exposure, excessive data exposure, and weak input validation.
- Evaluate usability, responsiveness, accessibility basics, cross-browser consistency, and error handling.
- Perform a basic, non-destructive performance test against an approved safe endpoint.
- Create a concise regression suite covering the most business-critical workflows.
- Implement at least three meaningful automated checks using Playwright with Python where the environment permits.
- Produce clear, reproducible, prioritized QA documentation suitable for developer review and release decision-making.

3. Scope

3.1 In Scope

- Registration and account creation flows.
- Login, logout, failed login behavior, session handling, and session persistence.
- User profile viewing and editing, including country and subscription information.
- Dashboard content and consistency of portfolio and asset values.
- Gold wallet: balance, buy, sell, validation, insufficient balance, and transaction history.
- Silver wallet: balance, buy, sell, validation, insufficient balance, and transaction history.
- ETF catalogue, search, details, prices, providers, risk level, expense ratio, performance, and allocation information.
- Country-based ETF eligibility for Germany, UAE, Saudi Arabia, USA, and other configured countries where possible.
- FREE versus PREMIUM access restrictions and prevention of bypass through direct API access.
- End-to-end ETF investment workflow including validation, confirmation, portfolio update, repeated submission, refresh, and browser back behavior.

- Transaction history accuracy, uniqueness, ownership, timestamps, status, and balance consistency.
- API functional and negative testing for available authentication, user, ETF, portfolio, investment, transaction, and insight endpoints.
- Basic security-oriented QA against explicitly provided test accounts and environments.
- Basic performance/load testing against an approved non-destructive endpoint.
- Desktop and mobile/responsive UI/UX validation.
- Cross-browser testing using Chrome plus Edge or Firefox where available.
- Critical-path regression testing.
- Basic Playwright + Python automation for at least three meaningful scenarios.
- Bug reporting, risk assessment, QA summary, evidence capture, and final demo preparation.

3.2 Out of Scope

- Destructive penetration testing, denial-of-service testing, credential theft, data deletion, or attempts to compromise production systems.
- Testing of accounts, user records, or environments not explicitly provided or authorized for the evaluation.
- Full-scale production load or stress testing.
- Deep source-code review, static application security testing, infrastructure penetration testing, or database exploitation.
- Formal compliance certification for PCI DSS, GDPR, AML/KYC, ISO standards, or other regulatory frameworks.
- Testing every possible browser, operating system, mobile device, language, currency, country, or permutation of business rules.
- Validation of third-party provider internals that are outside the accessible AURIX environment.
- Production transaction settlement or use of real customer funds.

4. Test Approach

A risk-based end-to-end approach will be used. High-risk workflows involving authentication, wallet balances, investments, eligibility, transaction integrity, authorization, and API access will be tested first. Positive, negative, boundary, usability, security-oriented, and consistency checks will be combined rather than treated as isolated activities. Evidence will be captured for significant failures and for selected passes where proof is important.

Test Type	Purpose	Priority
Functional	Verify core user journeys and business rules.	Very High
API	Validate service behavior, status codes, validation, and access control.	Very High
Security-oriented QA	Identify auth, authorization,	Very High

	IDOR, data exposure, and validation issues.	
Regression	Protect critical workflows after changes or retesting.	High
UI/UX	Assess usability, responsiveness, error states, and accessibility basics.	High
Cross-browser	Detect browser-specific behavior differences.	Medium
Performance	Measure basic response behavior under controlled safe load.	Medium
Automation	Demonstrate repeatable checks for critical flows.	High

5. Test Environment

The exact environment values will be recorded during execution. The following configuration will be used or updated with actual values before final submission.

Item	Planned / Actual
Operating system	Windows 11
Desktop browser 1	Google Chrome - version to be recorded
Desktop browser 2	Microsoft Edge or Mozilla Firefox - version to be recorded
Desktop viewport	Native desktop resolution / browser viewport to be recorded
Mobile/responsive viewport	Chrome DevTools responsive mode; representative mobile size to be recorded
Application URL	https://aurixapp.de or dedicated staging URL if supplied

API tool	Postman
Performance tool	Apache JMeter
Automation	Playwright + Python
Defect/evidence support	Screenshots, screen recording, browser DevTools, logs where available
Repository	GitHub or GitLab

6. Test Data

- Dedicated test user accounts only, including at least one standard/free account and a premium account if supplied or configurable.
- Valid and invalid email formats.
- Weak, valid, mismatched, and duplicate registration credentials.
- Country values including Germany, UAE, Saudi Arabia, USA, and any additional configured countries.
- Valid and invalid ETF identifiers and search terms.
- Investment amounts including zero, negative, decimal, boundary, large, and insufficient-balance values.
- Valid and invalid API IDs, missing parameters, malformed data, incorrect data types, expired/invalid authentication where safely reproducible.
- Two explicitly authorized test users for authorization/IDOR validation, if provided.

7. Assumptions and Constraints

- The evaluation environment is available and sufficiently stable during the 24-hour window.
- Any staging environment, test credentials, API documentation, or role-specific accounts provided by HOPn/AURIX are authorized for testing.
- Only non-destructive testing will be performed.
- If a feature is not implemented, inaccessible, or lacks required test data, the related test will be marked Blocked rather than Failed unless observable behavior is itself defective.
- Performance testing will only target an approved safe environment/endpoint and will remain small in scale.
- The 24-hour constraint requires prioritization; not every possible scenario or combination will be executed.
- Automation will focus on stable, meaningful user journeys. If technical limitations prevent execution, framework setup and representative executable code will be provided with the limitation documented.
- API security checks will be limited to explicitly authorized test accounts and non-destructive requests.

8. Risk Areas

Risk Area	Why It Matters	Impact	Test Priority
Authentication / session control	Unauthorized access could expose financial data or account functions.	Critical	P0
Authorization / IDOR	One user accessing another user's portfolio or transactions is a severe privacy and security failure.	Critical	P0
Financial transaction integrity	Duplicate, missing, or incorrect transactions can create monetary loss and trust issues.	Critical	P0
Portfolio / balance consistency	Incorrect balances or inconsistent values can mislead users and indicate data integrity defects.	High	P1
ETF eligibility / subscription enforcement	Restricted products may be exposed or purchasable by ineligible users.	High	P1
API validation and access control	Frontend controls can be bypassed if APIs do not independently enforce rules.	High	P1
Input validation	Invalid amounts, IDs, or malformed values may create errors or integrity issues.	High	P1
Performance bottlenecks	Slow critical endpoints degrade customer experience and	Medium	P2

	transaction confidence.		
Responsive / browser compatibility	Layout or control failures may block users on common devices.	Medium	P2

9. Entry Criteria

- Evaluation instructions received and scope understood.
- Application or staging URL is reachable.
- Required test accounts/credentials are available where applicable.
- Browser and testing tools are installed and functioning.
- API endpoints are discoverable through documentation, browser DevTools, or other authorized means.
- The tester understands the non-destructive restrictions and authorized boundaries of the exercise.

10. Exit Criteria

- At least 30 meaningful test cases have been documented, with priority coverage executed where the environment permits.
- Critical workflows have been assessed: registration, login, dashboard, wallets, ETF catalogue/eligibility, investment, portfolio, transactions, and logout.
- Significant defects have professional bug reports with reproducible steps and evidence where relevant.
- Available APIs have been tested with positive and negative scenarios and a Postman collection has been prepared.
- Basic security-oriented checks have been completed within authorized limits.
- A safe basic performance test has been executed or a justified blocker documented.
- At least three automated Playwright tests have been implemented or environment limitations clearly documented.
- Regression results are recorded as Passed, Failed, or Blocked.
- QA summary, top-five risk assessment, README, evidence, and final demo materials are completed.
- No unresolved Critical/P0 defect is omitted from the final QA summary.

11. Defect Classification

Severity	Definition	Typical Example	Priority Guidance
Critical	Severe security, financial, or system-wide failure with no acceptable workaround.	Unauthorized portfolio access; transaction corruption.	Usually P0

High	Major business function is broken or key controls can be bypassed.	Restricted ETF can be purchased; incorrect balance after investment.	Usually P1
Medium	Important behavior is incorrect but core service remains usable or workaround exists.	Poor validation, broken secondary workflow, significant responsive issue.	Usually P2
Low	Cosmetic, wording, minor alignment, or low-impact usability issue.	Spacing inconsistency or minor label problem.	Usually P3

12. Test Deliverables

- QA Test Plan.
- Minimum 30 structured test cases.
- Professional bug reports with evidence.
- Postman API collection.
- JMeter performance test and short result summary.
- Minimum 3 Playwright + Python automated tests.
- Critical workflow regression checklist and execution status.
- QA Test Summary Report.
- Top-five Risk Assessment.
- Screenshots, logs, and video evidence where relevant.
- GitHub/GitLab repository with organized project structure.
- README describing execution, results, limitations, and recommendations.
- 5-8 minute final demonstration video.

13. Planned Execution Sequence

1. Review scope, environment, credentials, and available documentation.
2. Confirm test environment and capture baseline screenshots.
3. Execute registration, login, logout, and session tests.
4. Test profile and dashboard consistency.
5. Test Gold and Silver wallet flows.
6. Test ETF catalogue, search, details, eligibility, and subscription controls.
7. Execute the complete investment flow and transaction/portfolio consistency checks.

8. Perform API functional and negative testing in Postman.
9. Perform authorized security-oriented API/UI checks.
10. Run desktop, mobile/responsive, and cross-browser checks.
11. Execute a safe JMeter performance test.
12. Implement and run at least three Playwright + Python automated tests.
13. Execute regression checklist for critical workflows.
14. Finalize bug reports, risk matrix, QA summary, README, evidence, and demo script.

14. Reporting and Status Conventions

Each executed test case will use one of the following statuses:

Status	Meaning
Passed	Actual result matches the expected result.
Failed	Actual result does not match the expected result and represents a defect.
Blocked	Test cannot be completed because of an environment, dependency, access, or implementation limitation.
Not Run	Test has been documented but not executed within the evaluation window.

15. Completion Statement

This plan is intentionally risk-based and proportionate to the 24-hour evaluation window. Priority will be given to security, financial correctness, authorization, API enforcement, critical end-to-end journeys, and clear evidence-based reporting. Lower-risk cosmetic coverage will be performed after critical business and technical risks have been assessed.